



School of Future Tech

Case Study Report

on

17.Digital Ride-Share Passenger Match Engine

by

Irene Koner

150096725146

1. Problem Statement

Modern ride-sharing platforms such as Uber and Ola handle thousands of drivers, passengers, ride requests, and route calculations every day. Managing this information efficiently is essential to provide fast service, minimize waiting time, and optimize travel costs.

The challenge is to design a system that can efficiently store driver information, process ride requests fairly, retrieve passenger locations quickly, rank high-demand areas, support rollback of administrative changes, and determine optimal routes between locations.

This project, RideNow, aims to simulate a ride-sharing management system using Data Structures and Algorithms (DSA) to improve operational efficiency and demonstrate real-world applications of fundamental DSA concepts.

2. Objectives

The primary objectives of the project are:

1. To efficiently manage driver information using a B-Tree.
2. To process ride requests fairly using a Queue (FIFO).
3. To maintain administrative change history using a Stack for rollback functionality.
4. To provide fast passenger location lookup using Hashing.
5. To rank city areas based on ride demand using Sorting algorithms.
6. To model the city transportation network using Graphs.
7. To determine the shortest travel path using Dijkstra's Algorithm.
8. To identify cost-effective routes by considering toll charges.
9. To allocate the nearest available driver to passengers.
10. To demonstrate practical applications of Data Structures and Algorithms in transportation systems.

3. Design / Architecture

The RideNow system consists of several interconnected modules.

Driver Management Module

This module stores and manages driver information using a B-Tree. It supports insertion, searching, and retrieval of driver records.

Ride Request Management Module

Ride requests are stored in a Queue and processed in the order they are received, ensuring fairness among users.

Location Lookup Module

Passenger locations are maintained using a Hash Map, allowing constant-time average lookup.

Administrative Module

Administrative modifications are stored in a Stack, enabling rollback of the most recent changes.

Demand Analysis Module

Area demand information is stored and sorted to identify high-demand regions where drivers should be allocated.

Route Optimization Module

The city road network is represented as a weighted Graph. Dijkstra's Algorithm is used to find the fastest route and toll-saving route between locations.

System Architecture Flow

```
graph TD; A[Passenger Request] --> B[Queue Processing]; B --> C[Location Lookup (Hash Map)]; C --> D[Driver Search (B-Tree)]; D --> E[Nearest Driver Selection]; E --> F[Route Optimization (Graph + Dijkstra)]; F --> G[Ride Completion];
```

Passenger Request
↓
Queue Processing
↓
Location Lookup (Hash Map)
↓
Driver Search (B-Tree)
↓
Nearest Driver Selection
↓
Route Optimization (Graph + Dijkstra)
↓
Ride Completion

4. Algorithm Description

1. B-Tree for Driver Management

The B-Tree stores driver records using Driver IDs as keys.

Steps:

1. Insert driver information into the B-Tree.
2. Search drivers using Driver ID.
3. Retrieve driver information efficiently.

Purpose: Provides efficient searching and insertion even when the number of drivers becomes large.

2. Stack for Administrative Rollback

The Stack stores administrative changes.

Steps:

1. Push each new administrative change onto the Stack.
2. When undo is requested, pop the latest change.
3. Restore the previous configuration.

Purpose: Implements Last-In-First-Out (LIFO) rollback functionality.

3. Queue for Ride Request Processing

Ride requests are stored inside a Queue.

Steps:

1. Insert new ride requests at the rear.
2. Process requests from the front.
3. Remove completed requests.

Purpose: Ensures First-In-First-Out (FIFO) processing and fairness.

4. Hash Map for Location Lookup

Passenger locations are stored using an unordered map.

Steps:

1. Store UserID → Location mapping.
2. Retrieve location using User ID.

Purpose: Provides $O(1)$ average lookup time.

5. Sorting for Busy Area Ranking

Areas are ranked according to ride demand.

Steps:

1. Store area demand values.
2. Sort areas in descending order.
3. Display rankings.

Purpose: Identifies high-demand locations for better driver distribution.

6. Graph Representation

The city road network is represented as a weighted graph.

Components:

- Vertices → Locations
- Edges → Roads
- Edge Weights → Travel Time and Toll Cost

Purpose: Provides efficient route representation.

7. Dijkstra's Algorithm

Dijkstra's Algorithm finds the shortest path between two locations.

Steps:

1. Initialize source node distance as 0.
2. Mark all other nodes as infinity.
3. Repeatedly select the nearest unvisited node.
4. Update neighboring distances.
5. Continue until the destination is reached.

Purpose: Finds the fastest route.

8. Smart Route Selection

The system compares:

- Fastest Route
- Toll-Saving Route

If toll savings are significant and travel time increase is small, the toll-saving route is selected.

Purpose: Balances travel cost and travel time.

5. Complexity Analysis

Operation	Algorithm	Time Complexity
Driver Search	B-Tree	$O(\log n)$
Driver Insert	B-Tree	$O(\log n)$
Admin Change Push	Stack	$O(1)$
Admin Undo	Stack	$O(1)$
Add Ride Request	Queue	$O(1)$
Process Ride Request	Queue	$O(1)$
Location Lookup	Hash Map	$O(1)$ Average
Busy Area Sorting	Sorting	$O(n \log n)$
Fastest Route	Dijkstra	$O((V+E) \log V)$
Toll-Saving Route	Modified Dijkstra	$O((V+E) \log V)$

Space Complexity

Structure	Space Complexity
B-Tree	$O(n)$
Stack	$O(n)$
Queue	$O(n)$
Hash Map	$O(n)$
Graph	$O(V + E)$

6. Screenshots of Execution

Program Main Menu

```
===== RideNow System =====
1. Show Drivers
2. Add Admin Change
3. Undo Admin Change
4. Add Ride Request
5. Location Lookup
6. Show Busy Areas
7. Process Ride Requests
8. Exit
Enter choice: █
```

Driver Directory Display

```
===== RideNow System =====
1. Show Drivers
2. Add Admin Change
3. Undo Admin Change
4. Add Ride Request
5. Location Lookup
6. Show Busy Areas
7. Process Ride Requests
8. Exit
Enter choice: 1

Driver Directory using B-Tree:
ID: 101 | Name: Aman | Location Node: 0 | Available: Yes
ID: 102 | Name: Ravi | Location Node: 2 | Available: Yes
ID: 103 | Name: Neha | Location Node: 4 | Available: Yes
ID: 104 | Name: Priya | Location Node: 5 | Available: Yes
ID: 105 | Name: Suresh | Location Node: 1 | Available: Yes
```

Adding Administrative Change and Undo Administrative Change


```

===== RideNow System =====
1. Show Drivers
2. Add Admin Change
3. Undo Admin Change
4. Add Ride Request
5. Location Lookup
6. Show Busy Areas
7. Process Ride Requests
8. Exit
Enter choice: 2
Enter admin change: Night fare increased by 10%
Admin change added.

===== RideNow System =====
1. Show Drivers
2. Add Admin Change
3. Undo Admin Change
4. Add Ride Request
5. Location Lookup
6. Show Busy Areas
7. Process Ride Requests
8. Exit
Enter choice: 2
Enter admin change: Zone boundary updated
Admin change added.

===== RideNow System =====
1. Show Drivers
2. Add Admin Change
3. Undo Admin Change
4. Add Ride Request
5. Location Lookup
6. Show Busy Areas
7. Process Ride Requests
8. Exit
Enter choice: 3
Undoing: Zone boundary updated
Rollback successful.

===== RideNow System =====
1. Show Drivers
2. Add Admin Change
3. Undo Admin Change
4. Add Ride Request
5. Location Lookup
6. Show Busy Areas
7. Process Ride Requests
8. Exit
Enter choice: 3
Undoing: Night fare increased by 10%
Rollback successful.

```

Location Lookup

```

===== RideNow System =====
1. Show Drivers
2. Add Admin Change
3. Undo Admin Change
4. Add Ride Request
5. Location Lookup
6. Show Busy Areas
7. Process Ride Requests
8. Exit
Enter choice: 5
Enter User ID: 201
User 201 is at location node 3

===== RideNow System =====
1. Show Drivers
2. Add Admin Change
3. Undo Admin Change
4. Add Ride Request
5. Location Lookup
6. Show Busy Areas
7. Process Ride Requests
8. Exit
Enter choice: 5
Enter User ID: 202
User 202 is at location node 5

```

Busy Area Ranking

```
===== RideNow System =====
1. Show Drivers
2. Add Admin Change
3. Undo Admin Change
4. Add Ride Request
5. Location Lookup
6. Show Busy Areas
7. Process Ride Requests
8. Exit
Enter choice: 6

Busy Area Ranking:
Bandra - Demand: 120
Dadar - Demand: 110
Thane - Demand: 90
Vashi - Demand: 70
Andheri - Demand: 50
```

Adding Ride Request

```
===== RideNow System =====
1. Show Drivers
2. Add Admin Change
3. Undo Admin Change
4. Add Ride Request
5. Location Lookup
6. Show Busy Areas
7. Process Ride Requests
8. Exit
Enter choice: 4
Enter Request ID: 1
Enter User ID: 201
Enter Pickup Node: 3
Enter Drop Node: 1
Ride request added successfully.

===== RideNow System =====
1. Show Drivers
2. Add Admin Change
3. Undo Admin Change
4. Add Ride Request
5. Location Lookup
6. Show Busy Areas
7. Process Ride Requests
8. Exit
Enter choice: 4
Enter Request ID: 2
Enter User ID: 205
Enter Pickup Node: 4
Enter Drop Node: 2
Ride request added successfully.
```

Processing Ride Request

```
===== RideNow System =====
1. Show Drivers
2. Add Admin Change
3. Undo Admin Change
4. Add Ride Request
5. Location Lookup
6. Show Busy Areas
7. Process Ride Requests
8. Exit
Enter choice: 7

Processing Ride Request ID: 1
Assigned Driver: Suresh
Selected Route Type: Fastest Route
Total Time: 5 minutes
Total Toll: Rs. 80

Processing Ride Request ID: 2
Assigned Driver: Neha
Selected Route Type: Fastest Route
Total Time: 10 minutes
Total Toll: Rs. 0
```

7. Results and Findings

The RideNow system successfully demonstrated the practical application of multiple Data Structures and Algorithms.

Findings

1. B-Tree enabled efficient storage and retrieval of driver information.
2. Queue ensured fair processing of ride requests.
3. Stack provided reliable rollback functionality.
4. Hash Maps allowed instant location lookup.
5. Sorting successfully ranked high-demand areas.
6. Graph representation effectively modeled the city network.
7. Dijkstra's Algorithm produced efficient routes.
8. Smart route selection reduced toll expenses while maintaining acceptable travel time.

The system successfully integrated multiple DSA concepts into a realistic ride-sharing scenario.

8. Conclusion

The RideNow project successfully demonstrates how Data Structures and Algorithms can be applied to solve real-world transportation and ride-sharing challenges.

The project integrates B-Tree, Stack, Queue, Hash Maps, Sorting, Graphs, and Dijkstra's Algorithm to efficiently manage drivers, passengers, ride requests, and route optimization. Through these techniques, the system achieves faster data retrieval, fair request processing, efficient driver allocation, and optimized route selection.

The project highlights the importance of choosing appropriate data structures and algorithms to improve system performance, scalability, and user experience in modern ride-sharing platforms.

9. Github Repository Link: <https://github.com/irenechk/RideNow.git>